arXiv:2510.08464v2 [cs.RO] 14 Oct 2026

Don't Run with Scissors ✂: Pruning Breaks VLA Models but They Can Be Recovered

Jason Jabbour[1,2,⋆] **Dong-Ki Kim**[1] **Max Smith**[1] **Jay Patrikar**[1] **Radhika Ghosal**[2]

Youhui Wang[2] **Ali Agha**[1] **Vijay Janapa Reddi**[2] **Shayegan Omidshafiei**[1]

[1]FieldAI [2]Harvard University

Abstract: Vision-Language-Action (VLA) models are a promising paradigm for field robotics, but their deployment is bottlenecked by strict edge SWaP-C constraints. Compression techniques such as quantization, distillation, and pruning are well-established for LLMs and VLMs. Pruning, however, remains particularly underexplored for VLA models. We surprisingly find that pruning VLA models yields drastic performance degradation and increased safety violations. Through spectral analysis, we identify structural properties of VLA weights that help explain this fragility. Based on this insight, we introduce **GLUESTICK**, a post-pruning recovery method that restores much of the original model's capability while preserving sparsity benefits. GLUESTICK computes a one-time corrective term by interpolating between the dense and pruned models in weight space. At inference, each pruned layer applies this correction to recover lost performance with minimal overhead. Our approach requires no additional training and is agnostic to the pruning algorithm. Across diverse VLA architectures and domains, GLUESTICK recovers up to 100% of the success rate and restores safety to near-dense levels, while saving up to 5 GB of memory and achieving up to 1.75× speedup. Additional material can be found at: **https://gluestick-vla.github.io/**.

Keywords: Vision-Language-Action (VLA) Models, Pruning, Edge Robotics

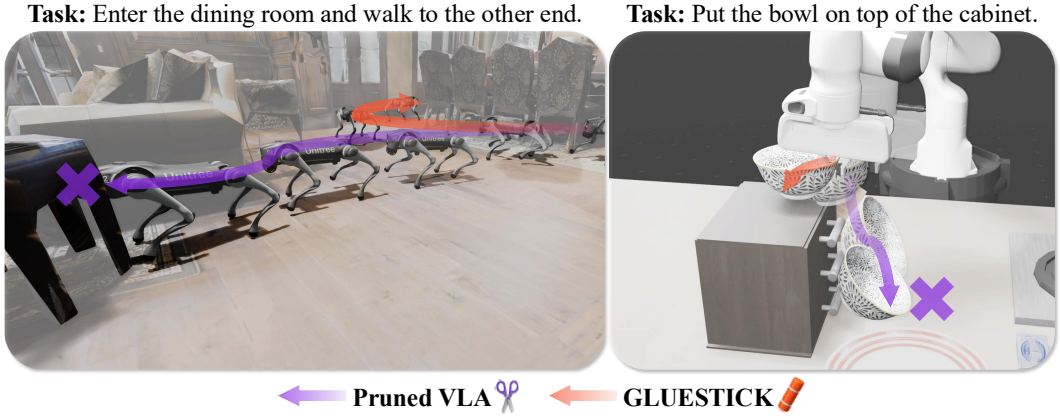


Figure 1: **VLA models break under pruning, and GLUESTICK fixes them.** Pruning methods unexpectedly cause task and safety failures in VLAs: colliding with an object in a navigation task **(left)**, or dropping a bowl in a manipulation task **(right)**. Our training-free post-pruning method, **GLUESTICK**, restores the lost functionality by adding a lightweight correction computed purely in weight space, while largely retaining the memory and speed benefits of pruning.

⋆Work conducted during Jason Jabbour's internship at FieldAI. Contact: jasonjabbour@g.harvard.edu

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

1 Introduction

Field robots operate in environments such as subterranean mines and tunnels [1, 2, 3, 4, 5, 6], disaster zones [7, 8, 9], offshore platforms [10, 11], and on the surface of other planets [12, 13, 14], where connectivity to off-board compute is unreliable or too slow for real-time control [15, 16, 17, 18]. As a result, field robots perform all computation onboard, on edge hardware governed by strict Size, Weight, Power, and Cost (SWaP-C) constraints [19, 20, 21, 22]. Recently, a new paradigm has emerged in robotics: Vision-Language-Action (VLA) models [23, 24, 25, 26]. Many current VLA models sit in the 7–8B parameter range, with some ranging up to 55–562B parameters [27, 28], and like the LLMs and VLMs that preceded them, VLAs follow scaling trends with the trend pointing upward [29, 30, 31]. The result is a footprint that exceeds the budget of widely deployed edge accelerators; for instance, OpenVLA 7B fails to load on the Jetson Orin NX entirely. For LLMs and VLMs, a mature toolbox of compression techniques has been developed: quantization [32], distillation [33], and pruning [34]. In robotics, quantization [23, 35] and distillation [36, 37] have been applied to VLAs, but pruning has not, leaving its impact on task success and safety unknown.

While pruning preserves accuracy for LLMs and VLMs [38, 34, 39], we surprisingly find that applying the same methods to VLAs leads to catastrophic degradation. In our experiments, recent pruning algorithms reduced task success rate on manipulation tasks from 85.2% to 0.0% and on a navigation task from 43.0% to 0.0%, while also increasing the number of safety violations. Despite this failure, pruning is too valuable to abandon. Pruning is particularly valuable for robotics because it reduces both memory and inference latency by removing weights entirely [40]. Quantization, distillation, and pruning are complementary and commonly composed together [41, 42, 43]; removing any one from the toolbox places a ceiling on how compact the deployed model can be. Restoring pruning to the VLA compression toolbox is a central goal of this work.

To this end, we introduce **GLUESTICK**, a post-pruning recovery method that restores signal lost during pruning while preserving the efficiency benefits of sparsity. Our spectral analysis reveals that VLA weights lack the concentrated spectral structure that makes LLMs and VLMs resilient to pruning, motivating GLUESTICK's design. GLUESTICK operates entirely in weight space, using the information discarded by pruning to nudge the model back toward more performant regions without any retraining. This is achieved by adding a lightweight correction term, computed from singular values in the gap between the dense and pruned weights. Our approach is pruning-agnostic and can be applied on top of any existing pruning algorithm. In doing so, GLUESTICK restores up to 100% of performance in some tasks, while maintaining the efficiency gains of structured sparsity by providing up to 5 GB memory savings and 1.75× speedup (Figure 1). We demonstrate that our method consistently recovers performance and improves safety across three VLA architectures, two widely used robotics benchmarks, multiple robot embodiments, and on real robotic systems.

Our contributions are as follows:

- **Empirical evidence of pruning collapse in VLAs.** We present the first systematic study showing that pruning, which is effective for LLMs and VLMs, causes near-complete collapse of the success rate in embodied VLA models, and an increase in safety violations.
- **Study of why VLAs differ from LLMs and VLMs under pruning.** Through spectral analysis, we identify structural properties of VLAs that could make them fragile to pruning.
- **An effective, training-free recovery method.** We propose GLUESTICK, a post-pruning recovery algorithm that restores lost signal after pruning using a low-rank, lightweight correction in weight space. GLUESTICK is pruning-algorithm-agnostic and requires no retraining.

2 Related Work

VLA Models. Recent work has developed VLA models that unify perception, language understanding, and decision-making into a single policy mapping multimodal inputs directly to robot actions. Prominent systems include OpenVLA [23], NaVILA [44], RT [45, 27], the π series [25, 46], PaLM-E [28], GR00T N1 [26], and CogACT [24], all of which are large end-to-end transformer-based policies with billions of parameters. The trend is toward richer inputs and outputs: SpatialVLA [47]

incorporates 3D scene information, while MolmoAct [48] and WorldVLA [49] extend outputs to include depth predictions or full world models, further growing model computational requirements. As these models scale, understanding how compression affects success and safety becomes critical.

Pruning for LLMs and VLMs. Pruning compresses models by setting a fraction of weights to zero [50]. Magnitude [51] removes small-magnitude weights, while Wanda [34] scores connections by activation statistics on calibration inputs. Both are training-free and computationally efficient. When applied to LLMs, pruning achieves strong accuracy retention. On LLaMA-2-70B [52], mean accuracy on the EleutherAI LM Harness [53] decreases by only 7.13% with Magnitude pruning and 2.94% with Wanda [34], despite removing 50% of weights under 2:4 structured sparsity. For VLMs, the impact is slightly more pronounced: Koike-Akino et al. [39] reported a 12.5% accuracy drop on LLaVA-7B, and Liang et al. [54] observed 9%–30% drops in LLaVA variants. No prior work has examined how pruning affects VLA success or safety, or how such impacts might be recovered.

Weight Modifications and Safety. Beyond pruning, other forms of weight adjustment have been widely studied, such as machine unlearning [55, 56] and fine-tuning [57]. These studies demonstrate that modifying weights can strongly affect downstream behavior: fine-tuning, for instance, has been shown to degrade LLM safety [58, 59], motivating post-finetuning frameworks to preserve alignment [60, 61]. Taken together, this line of work suggests that while pruning has been validated on LLMs, weight adjustments cannot be assumed to be benign in all contexts. For VLAs, where degraded behavior has physical consequences, the safety implications of pruning remain unstudied. Recovering from such damage in weight space remains an open problem that we address.

3 Motivation: Necessity of Pruning for Edge Devices on Field Robots

The gap between VLA resource requirements and the constraints of edge hardware is severe. We detail the severity of that gap and the distinct role pruning plays in addressing it.

Edge Hardware Constraints. Since field robots perform computation onboard, the hardware itself must fit within tight SWaP-C budgets [62, 63, 64]. Datacenter accelerators sit far outside this budget. An NVIDIA B200, for instance, consumes on the order of a kilowatt per GPU [65]. A typical field robot's battery pack stores only a few hundred watt-hours, meaning a single datacenter GPU would drain it in minutes, before powering any motors or sensors [63]. Instead, the field uses a family of edge accelerators, which consume on the order of tens of watts and can run for hours on the same battery. The NVIDIA Jetson Orin NX [66], AGX Orin [67], and Jetson Thor [68] are the dominant platforms for onboard compute in field robotics [69, 70, 71, 72, 73]. The tradeoff is memory and compute. The Orin NX, for example, provides only 8 to 16 GB shared between CPU and GPU; the OS and every other process on the robot draw from the same pool that the robotic algorithms must fit into. This is far below server-grade hardware, which may pair eight HGX B200 GPUs, each with 180 GB of dedicated memory, with terabytes of host RAM. The compute gap is equally stark. The Orin NX delivers just 4.8 FP16 TFLOPs [66], while a single B200 provides 4,500 FP16 TFLOPs [74, 75], nearly 1,000$\times$ more compute. For field robots, which are real-time systems that must react quickly to dynamic environments, this compute deficit directly limits how fast control loops can run.

Why Pruning Matters for Edge Robotics. Deploying VLAs on these devices requires reducing both memory footprint and inference latency. Pruning is especially attractive because it addresses both simultaneously by removing weights entirely rather than merely reducing their precision. When applied in hardware-friendly patterns such as structured "N:M" sparsity (e.g., 2:4), pruning reduces both parameter count and FLOPs. Modern GPUs exploit these patterns with specialized kernels that reduce memory traffic and multiply-accumulate operations (MACs); for example, NVIDIA's Sparse Tensor Cores and cuSPARSELt accelerate 2:4 sparse general matrix multiplications [40]. The reduced computation due to pruning not only enables acceleration and memory savings but also significantly cuts power consumption [76]. For devices operating under strict constraints on compute, memory, and energy, these benefits make pruning a critical tool for deploying VLAs at the edge. As discussed, quantization, distillation, and pruning are complementary; removing any one from the toolbox places a ceiling on how compact the deployed model can be. Our central goal is to bring pruning back into this toolbox for VLAs.

Model	Method	Success	Unsafe	Fits	Accel.
OpenVLA 7B	Dense	85.2%	33.4%	✗	✗
	Magnitude	0.0%	46.4%	✓	✓
	Wanda	0.0%	51.6%	✓	✓
NaVILA 8B	Dense	43.0%	23.0%	✗	✗
	Magnitude	0.0%	100.0%	✓	✓
	Wanda	0.0%	46.0%	✓	✓

Table 1: **Pruning impact on VLAs.** Success =% successful episodes; Unsafe=% with a safety violation; Fits= fits on Jetson Orin NX 16 GB; Accel.= accelerated inference.

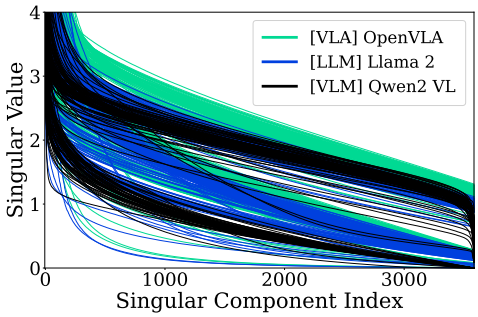


Figure 2: **Singular value spectra of weights**. VLA vs. LLM/VLM showing that VLA spectra are flatter and signal is more dispersed across the weight space.

4 GLUESTICK: Why Pruning Breaks VLA Models and How to Fix It

In this section, we first present our surprising finding that pruning can cause catastrophic degradation in VLA success and safety. We then introduce our new method, **GLUESTICK** (sin**G**ular va**LUE** **STI**t**C**hing), which glues pruned VLAs back to high task success rates and safe behaviors as in the original dense models. Additional details and pseudocode are available in Appendix A.

4.1 Pruning Severely Degrades VLA Performance

Pruning has substantially reduced memory usage in LLMs and VLMs, with minimal loss in accuracy (see Section 2). At the same sparsity levels commonly used in LLMs and VLMs, we observe surprisingly different outcomes on popular VLAs. Representative VLAs such as OpenVLA [23] and NaVILA [44], when pruned with Magnitude or Wanda, drop in task success rate to 0% (from 85.2% and 43.0%, respectively). Pruning also increases unsafe behavior: OpenVLA's unsafe-episode rate rises from 33.4% to 51.6%, and NaVILA's from 23.0% to 100.0% (see Table 1). These findings show that text and image-validated pruning techniques do not directly transfer to embodied control. Yet despite this, pruning remains essential as it allows models to fit on edge hardware and leverage sparse kernel acceleration (Table 1).

4.2 Why Pruning Breaks VLAs

To explain why pruning degrades VLAs far more than LLMs or VLMs, we ask whether their weight-space properties differ. Thus, we examine the singular value spectra of equal-sized layers within the language backbones of OpenVLA (VLA), LLaMA-2-7B (LLM) [52], and Qwen2-VL-7B-Instruct (VLM) [77, 78]. We display these spectra in Figure 2, where each line corresponds to the singular values of a single layer, obtained by computing the SVD of that layer's weight matrix and plotting its full set of singular values. In LLM and VLM models, the spectra are more anisotropic, evidenced by a steep initial drop followed by a long tail, which concentrates energy in a few dominant directions. This profile helps pruning, since removing small coefficients mainly trims low-energy di-

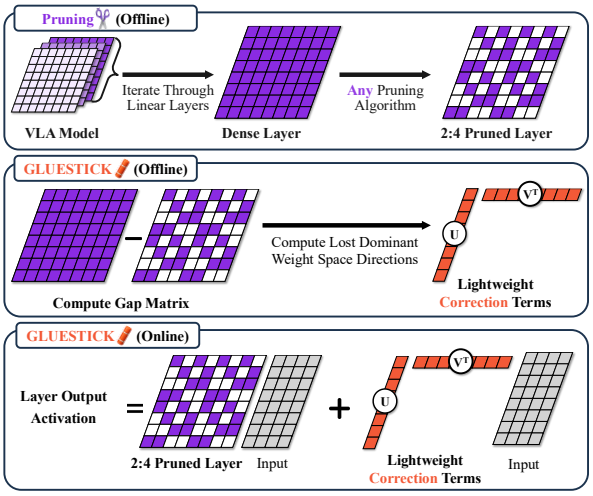


Figure 3: **Overview of GLUESTICK. (Top)** A VLA model is pruned with a standard algorithm (e.g., Wanda) to enforce 2:4 sparsity in linear layers. **(Middle)** Offline, the gap between the dense and pruned weights is computed and dominant lost directions are extracted via SVD, yielding lightweight corrections. **(Bottom)** At inference, these correction terms are applied alongside the pruned weights, adding back lost signal.

rections while leaving the principal subspaces intact. In contrast, VLA layers show a noticeably flatter decay, indicating energy spread across many directions. In this regime, even small-magnitude coefficients contribute to important subspaces, so pruning discards useful signal distributed throughout the matrix (see Appendix B for additional details). Based on this insight, our method in the following section explores the recovery of this lost information within the weight space.

4.3 The GLUESTICK Recovery Method for Pruned VLA Models

The space of pruning configurations is combinatorial, making optimal selection of weights to remove intractable. Heuristic methods such as Magnitude [51] and Wanda [34] sidestep the global optimization by scoring individual weights and pruning by score. While simple and efficient, these heuristics discard correlated weights under grouped sparsity constraints (e.g., 50% sparsity with 2:4 or 4:8 groups), which could be especially harmful for VLA models.

We propose **GLUESTICK**, a post-hoc, training-free recovery method that operates entirely in weight space and is agnostic to the pruning algorithm (see Figure 3). GLUESTICK requires only the original dense model and its pruned counterpart, and incurs a one-time offline cost; no additional training is required.

Specifically, for each linear layer with dense weight matrix $W_{\text{dense}} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and its pruned version W_{pruned} (fixed, preserving the original 2:4/4:8 pattern), we define the *gap matrix*:

$$W_{\text{gap}} = W_{\text{dense}} - W_{\text{pruned}}, \tag{1}$$

which captures lost information due to pruning. We then compute a truncated singular value decomposition (SVD) of the gap matrix:

$$W_{\text{gap}} = U \Sigma V^\top \approx U_r \Sigma_r V_r^\top, \tag{2}$$

keeping the top r singular components. By Eckart and Young [79], this is the *best rank-r* approximation to W_{gap} in Frobenius norm. For memory and speed, we fold the singular values into one term so that only two compact matrices need to be stored:

$$A = U_r \Sigma_r \in \mathbb{R}^{d_{\text{out}} \times r}, \quad B = V_r \in \mathbb{R}^{d_{\text{in}} \times r}. \tag{3}$$

During inference, GLUESTICK adds a lightweight correction around each pruned layer:

$$h(x) = W_{\text{pruned}} x + A(B^\top x), \tag{4}$$

which re-injects the dominant lost directions at low cost while leaving W_{pruned} unchanged, thereby preserving the efficiency gains of structured sparsity with a minimal overhead addition from the correction term. The extra compute from this correction is:

$$\underbrace{W_{\text{pruned}} x}_{\text{efficient sparse matmul}} + \underbrace{B^\top x}_{\mathcal{O}(d_{\text{in}}\, r)} + \underbrace{A(\,\cdot\,)}_{\mathcal{O}(d_{\text{out}}\, r)}, \tag{5}$$

or $\mathcal{O}((d_{\text{in}} + d_{\text{out}})r)$ on top of the sparse matrix-matrix multiplication (matmul), versus $\mathcal{O}(d_{\text{in}} d_{\text{out}})$ for the dense layer. Our correction adds only $(d_{\text{in}} + d_{\text{out}})r$ extra parameters per layer, which is small compared to $d_{\text{in}} d_{\text{out}}$ in the dense case. With $r \ll \min\{d_{\text{in}}, d_{\text{out}}\}$, GLUESTICK preserves the efficiency gains of structured 50% sparsity.

We refer to our method as GLUESTICK-r to indicate the chosen value of r. We note that the parameter r provides a dial between memory usage and recovery. Smaller values of r favor memory savings, while larger values prioritize recovery. In practice, integrating GLUESTICK into a model requires only a wrapper around pruned layers (see Appendix A for details).

5 Evaluation

Our experimental setup spans two benchmarks covering distinct robotics domains: manipulation and navigation. We evaluate three different VLA models on these tasks, with results measured using both task success and safety metrics. We structure our results around key questions, first presenting main findings on GLUESTICK then providing analysis through ablations and broader considerations.

Method	LIBERO (↑)				Mean (↑)
	Spat.	Obj.	Goal	Long	
Full Sparse	0.0	13.4	0.8	0.0	3.6
Sparse L.B.	17.3	25.8	20.5	6.2	17.5
% Sparse L.B.	15.2	25.2	21.2	5.7	16.8
GLUESTICK	**54.0**	**48.2**	**46.1**	**13.3**	**40.4**

Table 2: **Task success rate (%).** Results averaged across OpenVLA and WorldVLA. % Sparse L.B. uses the same VRAM as GLUESTICK.

Method	LIBERO (↓)				Mean (↓)
	Spat.	Obj.	Goal	Long	
Full Sparse	+18.2	+23.0	+1.4	+11.6	+13.6
Sparse L.B.	+9.0	+13.2	+1.8	+9.4	+8.4
% Sparse L.B.	+5.6	+14.4	+4.6	+5.2	+7.5
GLUESTICK	**+0.2**	**+1.2**	**+0.0**	**+4.6**	**+1.5**

Table 3: **Change in unsafe-episode rate (%) relative to Full Dense.** Lower values indicate fewer episodes with safety violations.

Method	Success (↑)	PL (m) (↓)	DG (m) (↓)
Full Dense	43.0	11.7	5.9
Full Sparse	0.0	17.6	9.5
Sparse L.B.	23.0	14.8	8.5
% Sparse L.B.	25.0	14.9	8.4
GLUESTICK	**44.0**	**11.9**	**5.9**

Table 4: **Navigation performance.** Success = task success rate (%). % Sparse L.B. uses the same VRAM as GLUESTICK. PL = Path Length. DG = Final Distance to Goal.

Method	ΔUnsafe (↓)
Full Dense	+0.0
Full Sparse	+23.0
Sparse L.B.	+2.0
% Sparse L.B.	+12.0
GLUESTICK	**−4.0**

Table 5: **Navigation safety.** ΔUnsafe is change in unsafe-episode rate (%) relative to Full Dense; lower is better.

5.1 Experimental Setup

Environments. We evaluate on two benchmarks: LIBERO [80] for manipulation (see Figure 1, right), which has four task suites testing manipulation skills with a Franka Panda arm, and VLN-CE-Isaac [44] for navigation (see Figure 1, left), which simulates legged robots traversing indoor environments. See Appendix C.1 for additional details.

Models. We study three VLA models: OpenVLA [23], a 7B-parameter generalist VLA for manipulation; WorldVLA [49], a 7B manipulation-oriented VLA targeting long-horizon consistency; and NaVILA [44], an 8B-parameter navigation-focused VLA. See Appendix C.3 for details.

Evaluation Metrics. We evaluate on two axes: task success, defined as binary per-episode completion, and safety, defined as the absence of harm to the robot or its surroundings [81, 82, 83]. For manipulation, we monitor risks such as joint-limit violations, collisions, unsafe object motion, and containment breaches; for navigation, we track collisions. See Appendix C.2.2 for details.

Baselines. We compare against three pruning strategies: Full Sparse, Sparse Language Backbone, and a Memory-Matched Sparse Language Backbone (a subset of backbone layers pruned to match GLUESTICK's memory overhead). We use Wanda with 50% 2:4 structured sparsity. Unless otherwise stated, GLUESTICK refers to GLUESTICK-500. See Appendix C.4 for details.

5.2 Main Results

Q1: *Does GLUESTICK recover the task success rate for pruned VLAs?*

GLUESTICK recovers the lost success rate by up to 100%. On navigation, GLUESTICK recovers 100% of the lost performance, matching the dense baseline at 44.0% (Table 4). On manipulation, pruning collapses the average success rate across the four LIBERO suites to just 3.6%. GLUESTICK recovers over 10× the pruned baseline's success rate, reaching 40.4%, with recovery strongest in the Spatial suite (from 0.0% to 54.0%). GLUESTICK also outperforms the memory-matched pruning baseline by over 2× (Table 2). See Appendix D.1 for more details.

GLUESTICK's recovery is particularly striking, restoring not only success but also trajectory efficiency. As seen in Table 4, GLUESTICK restores path length and final distance-to-goal to values nearly identical to those of the dense baseline. To appreciate the extent of this recovery, consider what pruning does without it. Pruning's failure is not a matter of less efficient paths; the robot's behavior is fundamentally altered. Without GLUESTICK, pruned agents frequently veer into en-

Model	Device	Method	Latency (↓)	Speedup (↑)
OpenVLA	NVIDIA Thor	Full Dense	796.50 ± 25.85 ms	—
		Full Sparse	434.47 ± 3.63 ms	$1.83\times$
		Sparse Language Backbone	439.45 ± 13.37 ms	$1.81\times$
		GLUESTICK-200	453.58 ± 31.29 ms	$1.75\times$
		GLUESTICK-500	537.49 ± 28.19 ms	$1.48\times$
WorldVLA	NVIDIA Thor	Full Dense	2816.57 ± 65.73 ms	—
		Sparse Language Backbone	1488.44 ± 57.27 ms	$1.89\times$
		GLUESTICK-200	1691.05 ± 69.35 ms	$1.67\times$
		GLUESTICK-500	1860.56 ± 48.23 ms	$1.51\times$
OpenVLA	NVIDIA Orin NX	Full Dense	Out of Memory	—
		Sparse Language Backbone	1546.37 ± 57.32 ms	Deploys†
		GLUESTICK-200	1790.99 ± 42.15 ms	Deploys†
		GLUESTICK-500	2085.96 ± 58.50 ms	Deploys†

Table 6: **Edge inference latency.** End-to-end inference on NVIDIA Jetson Thor and Orin NX. Pruning accelerates inference; GLUESTICK retains speedup with minimal overhead. † Since dense exceeds device memory, speedup not compared but still accelerated; sparsity enables deployment.

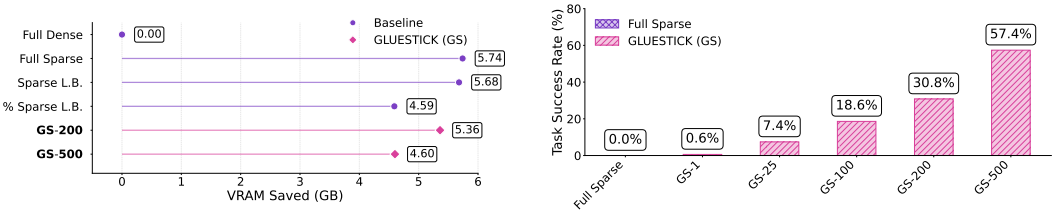


Figure 4: **Memory Savings and Rank Tradeoff. Left:** VRAM savings relative to Full Dense. All methods use NaVILA. **Right:** Rank ablation. All methods use OpenVLA and LIBERO Spatial.

tirely different rooms rather than approaching the goal. After Full Sparse pruning, mean path length increases by over 50% (11.7 to 17.6 m) and mean distance to goal by over 60% (5.9 to 9.5 m). This highlights that GLUESTICK not only mitigates pruning degradation but completely closes the gap to dense performance across both success and path-quality metrics. See Appendix D.2 for details.

Q2: *How well does GLUESTICK restore the safety of pruned VLAs?*

Across both manipulation and navigation, GLUESTICK preserves the safety profile of the dense policy, remaining near parity despite aggressive pruning. Tables 3 and 5 report changes in unsafe-episode rate relative to the dense baseline in manipulation and navigation, respectively. GLUESTICK yields 89% and 100% fewer unsafe episodes compared to the Full Sparse model for manipulation and navigation, respectively. That is, GLUESTICK maintains the safety profile of the original dense models. These results indicate that GLUESTICK restores dominant weight-space directions that carry both task-relevant and safety-critical signal, thereby preserving the safety of VLA models.

Q3: *Does GLUESTICK preserve the inference speedup of pruning?*

We evaluate inference latency on two edge devices: the NVIDIA Jetson Thor and the Jetson Orin NX (Table 6). On Thor, pruning with sparse kernels accelerates end-to-end inference by up to $1.83\times$–$1.89\times$. GLUESTICK adds only minimal overhead and still achieves a $1.75\times$ speedup for OpenVLA and $1.67\times$ for WorldVLA. Even GLUESTICK-500, which provides stronger task recovery, maintains a $1.48\times$ to $1.51\times$ speedup. On the Orin NX, the dense OpenVLA model cannot run at all, exceeding the device's memory limit. Conversely, GLUESTICK-200 successfully deploys on the Orin NX with only 245 ms of overhead relative to the sparse baseline. This overhead is minimal compared to the alternative: without GLUESTICK, the choice on the Orin NX is between a model that cannot load and a sparse model with severely degraded performance. See Appendix D.3.

Q4: *What are GLUESTICK's memory savings and how does r affect recovery–memory trade-off?*

GLUESTICK delivers substantial memory savings. As shown in Figure 4 (left), GLUESTICK-200 saves 5.36 GB of VRAM, offering memory savings within 0.38 GB of the fully sparse model, while recovering nearly the full dense success rate (Table 15). By comparison, a fully sparse NaVILA

Method	Dense	Dense/Int8	Sparse	Sparse/Int8	Sparse/Int8/GLUESTICK
Success Rate (%)	85.2	84.8	31.2	31.8	62.2

Table 7: **GLUESTICK composes with quantization.** Evaluating OpenVLA on LIBERO Spatial.

Method	0%	30%	50%	70%
Wanda	85.2	80.8	31.8	1.2
Wanda + GLUESTICK	–	**85.8**	**60.2**	**24.6**

Table 8: **GLUESTICK-500's recovery of VLA success rates across sparsities.** Evaluated on LIBERO Spatial. Results for the pruned language backbone of OpenVLA using Wanda.

Method	0%	30%	50%	70%
Wanda	93.88	93.38	88.16	21.30
Wanda + GLUESTICK	–	–	**92.95**	**90.89**

Table 9: **GLUESTICK-500's ability to recover pruned VLM performance.** Evaluated on 1000 DocVQA examples using Wanda-pruned Qwen2-VL-7B-Instruct VLM. Values show ANLS.

model achieves the maximum memory savings of 5.74 GB but suffers a 0% success rate. This illustrates the trade-off that GLUESTICK exposes through a single hyperparameter r as shown in Figure 4 (right), increasing r improves success-rate recovery at the cost of additional memory, giving practitioners a direct dial between efficiency and task performance. See Appendix D.1 and D.2 for more details.

Method	2:4	4:8
Magnitude	11.0	25.4
Magnitude + GLUESTICK	**48.8**	**61.8**

Table 10: **GLUESTICK is pruning-algorithm and sparsity-pattern agnostic.** Magnitude-pruned OpenVLA language backbone evaluated on LIBERO Spatial (values show success %).

5.3 Analysis

Q5: *How compatible is GLUESTICK with other compression methods?*

GLUESTICK composes well with other compression techniques. To demonstrate this, we combine `Int8` quantization with pruning and GLUESTICK. The results in Table 7 show that quantization has minimal impact on the success rate of both dense and sparse models; however, pruning degradation remains severe even after quantization. GLUESTICK recovers performance to 62.2% even on top of quantization, achieving our goal of making pruning a viable tool in the VLA compression toolbox.

We also show that GLUESTICK can be combined with VLA-specific compression techniques such as token pruning. Token pruning reduces the number of input tokens, while GLUESTICK operates on pruned weights, making the two complementary rather than competing. We ran experiments combining GLUESTICK with VLA-Pruner [84] on NVIDIA Thor and found that their speedups compound: GLUESTICK's 1.48× speedup rises to 1.64×, with an additional 0.12 GB of VRAM savings (VLA-Pruner targets latency rather than memory reduction). VLA-Pruner costs a 7% decrease in success due to token removal. This further shows GLUESTICK's composability.

Q6: *Is GLUESTICK agnostic to the pruning algorithm and sparsity pattern?*

To test whether GLUESTICK generalizes beyond the Wanda 2:4 setting used in our main results, we apply it to a Magnitude-pruned language backbone of OpenVLA under both 2:4 and 4:8 sparsity and evaluate on LIBERO Spatial (see Table 10). GLUESTICK improves success from 11.0% to 48.8% under 2:4 sparsity and from 25.4% to 61.8% under 4:8 sparsity, indicating that it is effective across both pruning algorithms and sparsity patterns.

Q7: *How does GLUESTICK perform at varying VLA sparsity levels?*

We evaluate GLUESTICK's capability to recover pruned VLA models under different unstructured sparsity ratios in Table 8. In this experiment, we prune the language backbone in OpenVLA and observe that GLUESTICK recovers performance effectively across sparsity levels.

Q8: *Does GLUESTICK generalize to restore pruned VLMs?*

We evaluate GLUESTICK's capability to recover the performance of a pruned VLM under various sparsity ratios in Table 9. We find strong generality, as GLUESTICK recovers VLM performance even at a harsh 70% sparsity, significantly improving ANLS from 21.30 to 90.89.

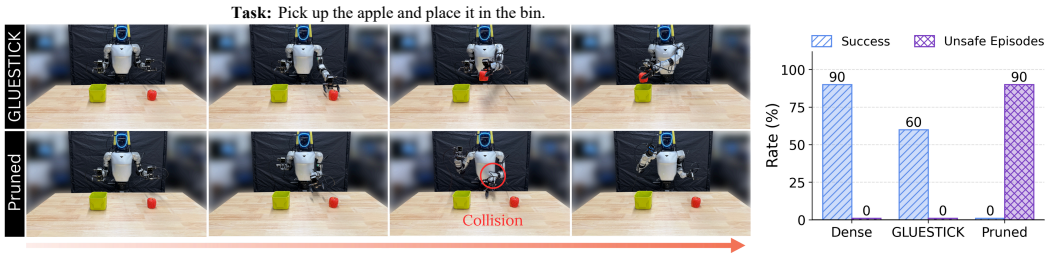


Figure 5: **Real-World Manipulation on Unitree G1.** With the pruned model, the G1's arm collides with its own body and the task fails, while with GLUESTICK the G1 successfully places the apple in the bin (**left**). Pruning results in 90% unsafe episodes and 0% task success, while GLUESTICK fully restores safety and reaches 60% success (**right**).

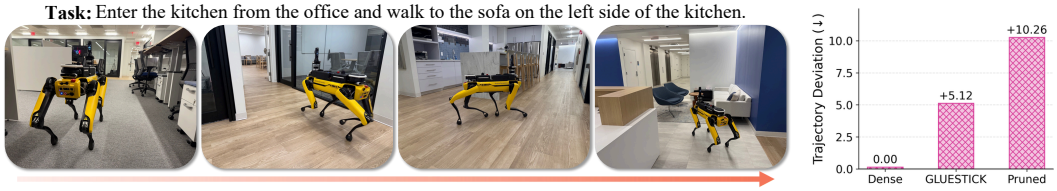


Figure 6: **Real-World Navigation on Boston Dynamics Spot.** Our real Spot robot navigating in a real office environment **(left)**. GLUESTICK reduces pruned NaVILA deviation by 50% **(right)**.

Q9: *How does GLUESTICK perform on real-world robots?*

We evaluate GLUESTICK on two real-world robotic platforms, a Unitree G1 humanoid for manipulation and a Boston Dynamics Spot for navigation. For manipulation, our setup (Figure 5, left) deploys GR00T N1.7 [85] on the G1, fine-tuned to pick up an apple and place it in a bin. We prune both the vision and language backbones of the VLA using Magnitude pruning at 50% 2:4 sparsity. We run 10 trials per model and apply GLUESTICK-1000 to all pruned layers, reporting results in Figure 5 (right). Pruning causes the model to fail completely, reducing success from 90% to 0% and producing unsafe behavior in 90% of episodes, with safety as defined in Appendix C.2.2. Interestingly, in nearly every episode, the pruned model exhibited self-collisions, collisions with the table, containment breaches in which the end effector moved beyond the table's edges, or a combination of these. We also observed the arms twisting into contorted, unnatural poses. In contrast, GLUESTICK fully restores safety and reaches 60% success.

For navigation, we record observations from a real Spot robot maneuvering through a real office environment (Figure 6, left). We prune NaVILA using Wanda at 50% 2:4 sparsity and replay the recorded observations through each model, measuring trajectory deviation, defined as the cumulative L1 difference in linear and angular velocity commands relative to the Full Dense model. GLUESTICK reduces this deviation by 50% relative to the pruned model (Figure 6, right).

6 Conclusion

We presented the first systematic study of pruning VLA models and showed that pruning severely degrades both task success and safety. To address this, we introduced GLUESTICK, a training-free, pruning-agnostic, and practical post-pruning recovery method that reintroduces lost directions due to pruning, restoring performance and safety while retaining efficiency. Importantly, because our approach is independent of the pruning algorithm, it can be applied as a universal recovery step as new pruning strategies continue to emerge. We hope this work lays the foundation for compression techniques that make VLAs deployable on resource-constrained robots.

Limitations. We designed GLUESTICK with a single hyperparameter so that applications can tune it based on their needs, but this introduces an inherent trade-off between success, memory, and latency. We also applied a uniform rank (r) across all layers. Future work includes reducing this trade-off and exploring adaptive rank selection across layers for further efficiency and recovery.

Acknowledgments

We thank Rumi Loghmani for help with the humanoid demonstration. Jason Jabbour is supported by the NSF GRFP under Grant No. DGE-2140743. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the NSF.

References

[1] A. Dixit, D. D. Fan, K. Otsu, S. Dey, A.-A. Agha-Mohammadi, and J. Burdick. Step: Stochastic traversability evaluation and planning for risk-aware navigation; results from the darpa subterranean challenge. *Field Robotics*, 4:182–210, 2024.

[2] I. D. Miller, F. Cladera, A. Cowley, S. S. Shivakumar, E. S. Lee, L. Jarin-Lipschitz, A. Bhat, N. Rodrigues, A. Zhou, A. Cohen, et al. Mine tunnel exploration using multiple quadrupedal robots. *IEEE robotics and automation letters*, 5(2):2840–2847, 2020.

[3] M. Tranzatto, T. Miki, M. Dharmadhikari, L. Bernreiter, M. Kulkarni, F. Mascarich, O. Andersson, S. Khattak, M. Hutter, R. Siegwart, et al. Cerberus in the darpa subterranean challenge. *Science Robotics*, 7(66):eabp9742, 2022.

[4] N. Hudson, F. Talbot, M. Cox, J. Williams, T. Hines, A. Pitt, B. Wood, D. Frousheger, K. L. Surdo, T. Molnar, et al. Heterogeneous ground and air platforms, homogeneous sensing: Team csiro data61's approach to the darpa subterranean challenge. *Field Robotics*, 2:595–636, 2022.

[5] S. Scherer, V. Agrawal, G. Best, C. Cao, K. Cujic, R. Darnley, R. DeBortoli, E. Dexheimer, B. Drozd, R. Garg, et al. Resilient and modular subterranean exploration with a team of roving and flying robots. *Field Robotics*, 2:678–734, 2022.

[6] M. F. Ginting, D. D. Fan, S.-K. Kim, M. J. Kochenderfer, and A.-A. Agha-Mohammadi. Semantic belief behavior graph: Enabling autonomous robot inspection in unknown environments. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7604–7610. IEEE, 2024.

[7] K. Nagatani, S. Kiribayashi, Y. Okada, K. Otake, K. Yoshida, S. Tadokoro, T. Nishimura, T. Yoshida, E. Koyanagi, M. Fukushima, et al. Emergency response to the nuclear accident at the fukushima daiichi nuclear power plants using mobile rescue robots. *Journal of Field Robotics*, 30(1):44–63, 2013.

[8] R. Murphy. Disaster robotics. intelligent robotics and autonomous agents series. *The MIT Press*, 2014.

[9] J. Delmerico, S. Mintchev, A. Giusti, B. Gromov, K. Melo, T. Horvat, C. Cadena, M. Hutter, A. Ijspeert, D. Floreano, et al. The current state and future outlook of rescue robotics. *Journal of Field Robotics*, 36(7):1171–1191, 2019.

[10] C. Gehring, P. Fankhauser, L. Isler, R. Diethelm, S. Bachmann, M. Potz, L. Gerstenberg, and M. Hutter. Anymal in the field: Solving industrial inspection of an offshore hvdc platform with a quadrupedal robot. In *Field and service robotics: results of the 12th international conference*, pages 247–260. Springer, 2021.

[11] O. Khatib, X. Yeh, G. Brantner, B. Soe, B. Kim, S. Ganguly, H. Stuart, S. Wang, M. Cutkosky, A. Edsinger, et al. Ocean one: A robotic avatar for oceanic discovery. *IEEE Robotics & Automation Magazine*, 23(4):20–29, 2016.

[12] B. Balaram, T. Canham, C. Duncan, H. F. Grip, W. Johnson, J. Maki, A. Quon, R. Stern, and D. Zhu. Mars helicopter technology demonstrator. In *2018 AIAA atmospheric flight mechanics conference*, page 0023, 2018.

[13] B. J. Morrell, M. S. da Silva, M. Kaufmann, S. Moon, T. Kim, X. Lei, C. Patterson, J. Uribe, T. S. Vaquero, G. J. Correa, et al. Robotic exploration of martian caves: Evaluating operational concepts through analog experiments in lava tubes. *Acta Astronautica*, 223:741–758, 2024.

[14] A. Patel, S. Karlsson, B. Lindqvist, C. Kanellakis, A.-A. Agha-Mohammadi, and G. Nikolakopoulos. Towards energy efficient autonomous exploration of mars lava tube with a martian coaxial quadrotor. *Advances in Space Research*, 71(9):3837–3854, 2023.

[15] G. Best, R. Garg, J. Keller, G. A. Hollinger, and S. Scherer. Resilient multi-sensor exploration of multifarious environments with a team of aerial robots. In *Robotics: Science and Systems (RSS)*, 2022.

[16] A. Agha, K. Otsu, B. Morrell, D. D. Fan, R. Thakker, A. Santamaria-Navarro, S.-K. Kim, A. Bouman, X. Lei, J. Edlund, et al. Nebula: Quest for robotic autonomy in challenging environments; team costar at the darpa subterranean challenge. *arXiv preprint arXiv:2103.11470*, 2021.

[17] S. Chinchali, A. Sharma, J. Harrison, A. Elhafsi, D. Kang, E. Pergament, E. Cidon, S. Katti, and M. Pavone. Network offloading policies for cloud robotics: a learning-based approach. *Autonomous Robots*, 45(7):997–1012, 2021.

[18] M. Alsolami, A. Bhattacharjee, and W. Shi. Evaluating the impact of network latency on the teleoperation of autonomous vehicles. In *Proceedings of the Tenth ACM/IEEE Symposium on Edge Computing*, pages 1–7, 2025.

[19] S. M. Neuman, B. Plancher, B. P. Duisterhof, S. Krishnan, C. Banbury, M. Mazumder, S. Prakash, J. Jabbour, A. Faust, G. C. de Croon, et al. Tiny robot learning: Challenges and directions for machine learning in resource-constrained robots. In *2022 IEEE 4th international conference on artificial intelligence circuits and systems (AICAS)*, pages 296–299. IEEE, 2022.

[20] T. Nguyen, S. S. Shivakumar, I. D. Miller, J. Keller, E. S. Lee, A. Zhou, T. Özaslan, G. Loianno, J. H. Harwood, J. Wozencraft, et al. Mavnet: An effective semantic segmentation micro-network for mav-based tasks. *IEEE Robotics and Automation Letters*, 4(4):3908–3915, 2019.

[21] O. Ennasr, B. M. Dodd, M. A. Paquette, C. C. Ellison, and G. P. Glaspell. Low size, weight, power, and cost (swap-c) payload for autonomous navigation and mapping on an unmanned ground vehicle. 2023.

[22] S. Krishnan, Z. Wan, K. Bhardwaj, N. Jadhav, A. Faust, and V. J. Reddi. Roofline model for uavs: A bottleneck analysis tool for onboard compute characterization of autonomous unmanned aerial vehicles. In *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 162–174. IEEE, 2022.

[23] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. 2024. doi: 10.48550/arXiv.2406.09246. URL https://arxiv.org/abs/2406.09246.

[24] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, M. Liao, F. Wei, Y. Deng, S. Xu, Y. Zhang, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024.

[25] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. pi0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[26] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. GR00T N1: an open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[27] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.

[28] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, et al. Palm-e: An embodied multimodal language model. 2023.

[29] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[30] T. Pearce, T. Rashid, D. Bignell, R. Georgescu, S. Devlin, and K. Hofmann. Scaling laws for pre-training agents and world models. *arXiv preprint arXiv:2411.04434*, 2024.

[31] N. Agarwal, A. Ali, M. Bala, Y. Balaji, E. Barker, T. Cai, P. Chattopadhyay, Y. Chen, Y. Cui, Y. Ding, et al. Cosmos world foundation model platform for physical ai. *arXiv preprint arXiv:2501.03575*, 2025.

[32] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in neural information processing systems*, 35: 30318–30332, 2022.

[33] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[34] M. Sun, Z. Liu, A. Bair, and Z. Kolter. A simple and effective pruning approach for large language models. In *International Conference on Learning Representations*, volume 2024, pages 4942–4964, 2024.

[35] J. Zhang, Y. Hsieh, Z. Wan, H. Lin, X. Wang, Z. Wang, Y. Lei, and M. Zhang. Quantvla: Scale-calibrated post-training quantization for vision-language-action models. *arXiv preprint arXiv:2602.20309*, 2026.

[36] Z. Ravichandran, I. Hounie, F. Cladera, A. Ribeiro, G. J. Pappas, and V. Kumar. Distilling on-device language models for robot planning with minimal human intervention. *arXiv preprint arXiv:2506.17486*, 2025.

[37] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg. Consistency policy: Accelerated visuomotor policies via consistency distillation. *arXiv preprint arXiv:2405.07503*, 2024.

[38] E. Frantar and D. Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International conference on machine learning*, pages 10323–10337. PMLR, 2023.

[39] T. Koike-Akino, J. Liu, and Y. Wang. mumoe: Test-time pruning as micro-grained mixture-of-experts. *arXiv preprint arXiv:2505.18451*, 2025.

[40] A. Mishra, J. Albericio Latorre, J. Pool, D. Stosic, D. Stosic, G. Venkatesh, C. Yu, and P. Micikevicius. Accelerating sparse deep neural networks, 2021. URL https://arxiv.org/abs/2104.08378.

[41] Z. Xu, A. Gupta, T. Li, O. Bentham, and V. Srikumar. Beyond perplexity: Multi-dimensional safety evaluation of llm compression. *arXiv preprint arXiv:2407.04965*, 2024.

[42] S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015.

[43] S. Muralidharan, S. Turuvekere Sreenivas, R. Joshi, M. Chochowski, M. Patwary, M. Shoeybi, B. Catanzaro, J. Kautz, and P. Molchanov. Compact language models via pruning and knowledge distillation. *Advances in Neural Information Processing Systems*, 37:41076–41102, 2024.

[44] A.-C. Cheng, Y. Ji, Z. Yang, Z. Gongye, X. Zou, J. Kautz, E. Bıyık, H. Yin, S. Liu, and X. Wang. Navila: Legged robot vision-language-action model for navigation. 2025. doi: 10.48550/arXiv.2412.04453. URL https://arxiv.org/abs/2412.04453.

[45] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.

[46] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[47] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, et al. Spatialvla: Exploring spatial representations for visual-language-action model. *arXiv preprint arXiv:2501.15830*, 2025.

[48] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee, et al. Molmoact: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025.

[49] J. Cen, C. Yu, H. Yuan, Y. Jiang, S. Huang, J. Guo, X. Li, Y. Song, H. Luo, F. Wang, D. Zhao, and H. Chen. Worldvla: Towards autoregressive action world model. 2025. doi:10.48550/arXiv.2506.21539. URL https://arxiv.org/abs/2506.21539.

[50] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang. A survey on model compression for large language models. *Transactions of the Association for Computational Linguistics*, 12:1556–1577, 2024.

[51] S. Han, J. Pool, J. Tran, and W. Dally. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems*, 28, 2015.

[52] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[53] L. Gao, J. Tow, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, K. McDonell, N. Muennighoff, J. Phang, L. Reynolds, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou. A framework for few-shot language model evaluation. Version v0.0.1, September 2021.

[54] Y. Liang, Z. Wang, X. Xu, J. Zhou, and J. Lu. Efficientllava: Generalizable auto-pruning for large vision-language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 9445–9454, 2025.

[55] A. Golatkar, A. Achille, and S. Soatto. Eternal sunshine of the spotless net: Selective forgetting in deep networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9304–9312, 2020.

[56] Z. Liu, G. Dou, Z. Tan, Y. Tian, and M. Jiang. Towards safer large language models through machine unlearning. *arXiv preprint arXiv:2402.10058*, 2024.

[57] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.

[58] S. Yang, Q. Zhang, Y. Liu, Y. Huang, X. Jia, K. Ning, J. Yao, J. Wang, H. Dai, Y. Song, et al. AsFT: Anchoring safety during llm fine-tuning within narrow safety basin. *arXiv preprint arXiv:2506.08473*, 2025.

[59] T. Huang, G. Bhattacharya, P. Joshi, J. Kimball, and L. Liu. Antidote: Post-fine-tuning safety alignment for large language models against harmful fine-tuning attack. In *Forty-second International Conference on Machine Learning*.

[60] A. Djuhera, S. R. Kadhe, F. Ahmed, S. Zawad, and H. Boche. SafeMERGE: Preserving safety alignment in fine-tuned large language models via selective layer-wise model merging. *arXiv preprint arXiv:2503.17239*, 2025.

[61] C.-Y. Hsu, Y.-L. Tsai, C.-H. Lin, P.-Y. Chen, C.-M. Yu, and C.-Y. Huang. Safe lora: The silver lining of reducing safety risks when finetuning large language models. *Advances in Neural Information Processing Systems*, 37:65072–65094, 2024.

[62] S. M. Neuman, B. Plancher, T. Bourgeat, T. Tambe, S. Devadas, and V. J. Reddi. Robomorphic computing: a design methodology for domain-specific accelerators parameterized by robot morphology. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 674–686, 2021.

[63] J. Jabbour and V. Janapa Reddi. Generative ai agents in autonomous machines: A safety perspective. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, pages 1–13, 2024.

[64] J. Jabbour, S. Neuman, M. Mazumder, C. Banbury, S. Prakash, B. Plancher, and V. Reddi. Closing the sim-to-real gap for ultra-low-cost resource-constrained quadruped robot platforms. In *3rd Workshop on Closing the Reality Gap in Sim2Real Transfer for Robotics*, 2022.

[65] NVIDIA. *DGX B200 Datasheet*, 2025. URL https://resources.nvidia.com/en-us-dgx-systems/dgx-b200-datasheet?ncid=no-ncid. Accessed: 2025-09-25.

[66] NVIDIA Corporation. Jetson Orin NX Series Modules Data Sheet. https://developer.nvidia.com/downloads/jetson-orin-nx-module-series-data-sheet, 2022. DS-10712-001 v1.7. Accessed: 2026-05-27.

[67] NVIDIA Corporation. *NVIDIA Jetson AGX Orin Technical Brief*. NVIDIA, 2021. URL https://www.nvidia.com/content/dam/en-zz/Solutions/gtcf21/jetson-orin/nvidia-jetson-agx-orin-technical-brief.pdf.

[68] NVIDIA Corporation. *Jetson Thor Series Modules Datasheet*. NVIDIA, 2025. URL https://developer.download.nvidia.com/assets/embedded/secure/jetson/thor/docs/Jetson_Thor_Series_Modules_Datasheet_DS-11945-001v1.4.pdf. Version DS-11945-001 v1.4.

[69] S. Park, H. Kim, W. Jeon, J. Yang, B. Jeon, Y. Oh, and J. Choi. Quantization-aware imitation-learning for resource-efficient robotic control. *arXiv preprint arXiv:2412.01034*, 2024.

[70] P. Liu, C. Feng, Y. Xu, Y. Ning, H. Xu, and S. Shen. Omninxt: A fully open-source and compact aerial robot with omnidirectional visual perception. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 10605–10612. IEEE, 2024.

[71] L. Rey, A. M. Bernardos, A. D. Dobrzycki, D. Carramiñana, L. Bergesio, J. A. Besada, and J. R. Casar. A performance analysis of you only look once models for deployment on constrained computational edge devices in drone applications. *arXiv preprint arXiv:2502.15737*, 2025.

[72] NVIDIA Corporation. Introducing nvidia jetson thor, the ultimate platform for physical ai, Aug. 2025. URL https://developer.nvidia.com/blog/introducing-nvidia-jetson-thor-the-ultimate-platform-for-physical-ai/. NVIDIA Technical Blog, accessed 2026-05-25.

[73] The Robot Report. Nvidia jetson thor bring 2k teraflops of ai compute to robots. Aug. 2025. URL https://www.therobotreport.com/nvidia-jetson-thor-brings-2k-teraflops-of-ai-compute-to-robots/. Accessed 2026-05-25.

[74] NVIDIA Corporation. NVIDIA Blackwell B200 GPU Datasheet. https://www.primeline-solutions.com/media/categories/server/nach-gpu/nvidia-hgx-h200/nvidia-blackwell-b200-datasheet.pdf, 2024. Accessed: 2026-05-27.

[75] Cornell Virtual Workshop. Horizon GPUs: Blackwell B200 SM Architecture. https://cvw.cac.cornell.edu/gpu-architecture/horizon-gpus-blackwell-b200/b200_sm, 2025. Accessed: 2026-05-27.

[76] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally. Eie: Efficient inference engine on compressed deep neural network. *ACM SIGARCH Computer Architecture News*, 44(3):243–254, 2016.

[77] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[78] Qwen Team. Qwen2-VL-7B-Instruct. https://huggingface.co/Qwen/Qwen2-VL-7B-Instruct, 2024. Accessed: 2026-10-13.

[79] C. Eckart and G. Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936. doi:10.1007/BF02288367.

[80] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. 2023. doi:10.48550/arXiv.2306.03310. URL https://arxiv.org/abs/2306.03310.

[81] G. Dulac-Arnold, D. Mankowitz, and T. Hester. Challenges of real-world reinforcement learning. *arXiv preprint arXiv:1904.12901*, 2019.

[82] Y. Geng, J. Ji, Y. Chen, L. Yang, and Y. Yang. A massively parallel benchmark for safe dexterous manipulation, 2023. URL https://openreview.net/forum?id=k2Ml8FGtJZp.

[83] D. Morton and M. Pavone. Safe, task-consistent manipulation with operational space control barrier functions. *arXiv preprint arXiv:2503.06736*, 2025.

[84] Z. Liu, Y. Chen, H. Cai, T. Lin, S. Yang, Z. Liu, and B. Zhao. VLA-Pruner: Temporal-aware dual-level visual token pruning for efficient vision-language-action inference. *arXiv preprint arXiv:2511.16449*, 2025.

[85] NVIDIA. GR00T N1.7: Open foundation model for generalist robots. https://github.com/NVIDIA/Isaac-GR00T, 2026. Model: `nvidia/GR00T-N1.7-3B`. Accessed: 2026-10-13.

[86] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee. Beyond the nav-graph: Vision-and-language navigation in continuous environments. In *European Conference on Computer Vision*, pages 104–120. Springer, 2020.

[87] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra. Habitat: A platform for embodied ai research. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9339–9347, 2019. doi:10.1109/ICCV.2019.00943. URL https://openaccess.thecvf.com/content_ICCV_2019/html/Savva_Habitat_A_Platform_for_Embodied_AI_Research_ICCV_2019_paper.html.

[88] P. Anderson, Q. Wu, D. Teney, J. Bruce, M. Johnson, N. Sünderhauf, I. Reid, S. Gould, and A. Van Den Hengel. Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3674–3683, 2018.

[89] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 11975–11986, 2023.

[90] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.

[91] C. Team. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818*, 2024.

[92] P. Esser, R. Rombach, and B. Ommer. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12873–12883, 2021.

[93] J. Lin, H. Yin, W. Ping, P. Molchanov, M. Shoeybi, and S. Han. Vila: On pre-training for visual language models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 26689–26699, 2024.

[94] H. Liu, C. Li, Y. Li, and Y. J. Lee. Improved baselines with visual instruction tuning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 26296–26306, 2024.

[95] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

Appendix

A GLUESTICK Details

Our method can be implemented in just a few lines of code.

Algorithm 1 PyTorch code for GLUESTICK (Offline)

```
# Compute and Store GLUESTICK Correction terms for every linear layer

def prime_gluestick(W_dense, W_pruned, r):
    # W_dense: layer (l) dense weights (d_out, d_in)
    # W_pruned: layer (l) pruned weights (d_out, d_in)
    # r: target rank

    W_gap = W_dense - W_pruned
    U, S, Vh = torch.linalg.svd(W_gap)

    U_r   = U[:, :r]
    S_r   = S[:r]
    V_r   = Vh[:r, :].T
    A     = U_r * S_r.unsqueeze(0)
    B     = V_r

    return {"A": A, "B": B}
```

In the offline stage (Algorithm 1), we iterate through the dense and pruned weights of each linear layer, compute the correction terms, and store them.

Algorithm 2 PyTorch code for GLUESTICK (Online)

```
class GLUESTICKWrap(nn.Module):
    def __init__(self, pruned_linear_layer, A, B):
        super().__init__()
        self.pruned_linear = pruned_linear_layer
        self.A = A
        self.B = B

    def forward(self, x):
        # Efficient Sparse MatMul
        y = F.linear(
            x,
            self.pruned_linear.weight,
            self.pruned_linear.bias
        )
        # Compute GLUESTICK Correction
        correction = self.A @ (self.B.T @ x)
        return torch.add(y, correction)

# Load Pruned Model
model = load_pruned_model()
# Load GLUESTICK correction terms for every linear layer
correction_terms = load_correction_terms()
# Apply GLUESTICK to all pruned linear layers in the model
model = apply_gluestick(model, correction_terms)
```

In the online stage (Algorithm 2), we load the pruned model along with the saved correction terms and wrap each pruned linear layer with GLUESTICK to enable corrected inference.

B Spectral Analysis

To compare LLM and VLA spectra, we examine the same language backbone pre/post VLA fine-tuning. OpenVLA's language backbone *is* a fine-tuned LLaMA-2, so both the OpenVLA and LLaMA-2 curves in Figure 2 show the same layers pre/post VLA fine-tuning. We quantify spectra with AUC/k, where k is the number of singular values of a layer. Since the curves decay monotonically, a higher AUC/k indicates slower decay, with more energy in later components. We observe that fine-tuning shifts the spectra of both the language backbone (1.32 to 1.55) and the vision encoders (0.84 to 0.92; DINOv2+SigLIP).

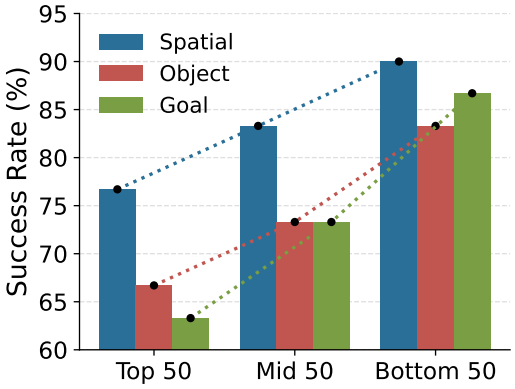


Figure 7: **Correlation of layer-wise spectra with pruning sensitivity.** Pruning the top, middle, and bottom 50 layers ranked by AUC/k change pre/-post VLA fine-tuning hurts success most, moderately, and least, respectively.

We also examine the relationship between layer-wise spectra and pruning sensitivity. In our experiments, layers with more dispersed spectra tend to be more sensitive to pruning (Figure 7). On LIBERO Object, GLUESTICK recovers 16.7% for the top 50 AUC/k-changed layers but only 3.3% for the bottom 50, suggesting that layers with larger spectral change may benefit more from correction.

C Experimental Setting

C.1 Environments

Manipulation. We evaluate on LIBERO [80], a benchmark designed to test embodied manipulation skills inspired by human activities (see Figure 1, right). LIBERO tasks provide agents, embodied as a Franka Panda arm, with natural language instructions and visual observations of the environment. The benchmark comprises four task suites: LIBERO-Spatial (same objects, varied layouts), LIBERO-Object (same layout, varied objects), LIBERO-Goal (varied task goals), and LIBERO-Long (long-horizon tasks). Our manipulation evaluation covers all 10 tasks from each of the four LIBERO suites, with each task repeated 50 times, resulting in 2,000 total episodes.

Navigation. We evaluate on the VLN-CE-Isaac benchmark [44], which simulates legged robots (e.g., the Unitree Go2 quadruped and the H1 humanoid) traversing indoor environments to reach goal locations (see Figure 1, left). Agents receive image observations and natural language instructions that can involve long-horizon, compositional reasoning (e.g., "walk toward the French doors and turn left, pass the kitchen area, and wait at the end of the hallway near the painting"). The robot executes velocity commands (e.g., move forward 0.75 m, turn right 15°), which are generated by a VLA model. VLN-CE-Isaac builds on VLN-CE [86], which itself is based on the Habitat simulator [87]. Habitat provides photorealistic 3D environments and physics-based simulation for embodied AI, moving beyond the original VLN task that used MatterPort3D panoramas represented as discrete navigation graphs [88]. Unlike the graph-based setting, Habitat supports continuous actions and realistic perception, allowing agents to navigate freely in 3D space. However, Habitat does not simulate robot embodiment. For instance, agents can move through unrealistic gaps (e.g., 10 cm between two sofas) that would be infeasible for legged robots. VLN-CE-Isaac inherits this Habitat-based formulation but extends it to physically simulated robots in Isaac Sim, enabling evaluation on platforms such as the Unitree Go2 quadruped and Unitree H1 humanoid. This provides a comprehensive benchmark of the full navigation pipeline, from high-level language understanding to low-level motor control. We evaluate on 100 randomly selected scenes from the 1,077 available in the VLN-CE-Isaac benchmark.

Hardware. Unless stated otherwise, experiments used an NVIDIA L40S GPU with 48 GB of VRAM.

C.2 Evaluation Metrics

C.2.1 Success

Success captures whether the agent achieves the stated goal; we report binary per-episode success where an episode is successful if the agent completes the objective, and unsuccessful otherwise.

C.2.2 Safety

Safety captures whether the agent achieves its goal without causing harm to itself or the environment. Following prior robotics safety work [81, 82, 83], we operationalize safety as the absence of harm caused to the robot or its surroundings. We monitor robot- and environment-centered risks.

Safety in Navigation. For navigation tasks, we use collisions as the primary safety metric. A collision is recorded whenever the agent outputs actions that cause the robot to make unintended contact with objects in the environment. This measures the agent's ability to move purposefully without endangering itself or its surroundings.

Safety in Manipulation. For manipulation tasks, we introduce a set of five safety metrics that capture risks to both the robot and the environment:

- **Joint limit violations:** Occur when the agent outputs actions that drive joint angles close to or beyond their mechanical limits, which can cause long-term wear or physical damage to the robot's actuators.
- **Arm collisions:** Measured when any part of the robot arm (excluding the end effector) makes unintended contact with the environment, potentially harming both the robot and external objects.
- **Object velocities:** We track the velocities of manipulated objects as a proxy for physical stability, penalizing outcomes where objects are flung, dropped, or otherwise move unsafely.
- **End-effector containment:** We enforce that the end effector remains within a bounded three-dimensional workspace region. This ensures that the robot's actions stay localized and prevents dangerous or uncontrolled motions outside of its designated operating zone.
- **Whole-body containment:** Similarly, we verify that the robot's entire body remains within a global containment region. Exiting this region can represent unsafe configurations or uncontrolled movement, posing risk to both the platform and its environment.

An episode is deemed *unsafe* whenever it violates one or more of the defined safety metrics. We use the following thresholds: joint limit violations occur if a joint comes within 0.1% of its limits; object motion is unsafe if velocity exceeds 1.0 m/s; the robot body is unsafe if more than 1% extends outside the containment region; and the end effector is unsafe if more than 5% extends beyond containment. Containment regions are computed from the ground-truth dataset.

C.3 Models

OpenVLA. OpenVLA [23] is a 7B-parameter generalist VLA for manipulation, built on the LLaMA-2 7B language backbone [52] with SigLIP [89] and DINOv2 [90] transformer-based vision encoders. It takes an image and a natural language instruction as input, and autoregressively outputs a 7D low-level end-effector pose and gripper open/close commands. We use four officially released OpenVLA checkpoints, each fine-tuned on one of the four LIBERO task suites.

WorldVLA. WorldVLA [49] is a 7B manipulation-oriented VLA that targets long-horizon consistency via an autoregressive action world-modeling objective. It is initialized from the 7B Chameleon

vision–language model [91] with a convolution-based VQ-GAN vision encoder [92]. WorldVLA ingests the current image observation, a sequence of history images, and a natural language instruction, and generates 7D low-level end-effector poses and gripper open/close commands in action chunks. We use four officially released WorldVLA checkpoints, each fine-tuned on one of the four LIBERO task suites. We ran experiments with the default action chunking of 25; however, we observed that pruning leads to sometimes meaningless token outputs under this setting causing invalid actions for the robot to execute. For this reason, we set the action chunk size to 1 for success and safety evaluations, as this produces the most reliable comparison across methods. For inference speed experiments, we use the default action chunk size of 25. WorldVLA also allows varying the number of history images; we adopt the default configuration of one history image with the current image.

NaVILA. NaVILA [44] is an 8B-parameter, navigation-focused VLA designed for legged robots. It is built on the VILA vision–language model [93], which combines a ViT-based visual encoder with a language backbone inspired by LLaVA's architecture [94], but pre-trained on a unique mixture of data. The model consumes the current egocentric image, a set of history frames, and natural language instructions, and outputs velocity commands executed by a locomotion controller.

C.4 Baselines

We consider three pruning strategies: (i) Full Sparse, where all linear layers except the model head are pruned with 50% 2:4 structured sparsity using Wanda; (ii) Sparse Language Backbone, where only the language backbone is pruned; and (iii) a Memory-Matched Sparse Language Backbone, which prunes a subset of backbone layers while maintaining 50% 2:4 sparsity in each pruned layer, to provide a fair comparison to GLUESTICK's memory overhead. We use a 2:4 structured sparsity across the main experiments, as this level of pruning is necessary to realize improvements from hardware-efficient sparse kernels. We choose Wanda as our base pruning algorithm because it represents the state of the art and is computationally efficient, requiring minimal cost during pruning.

D Main Results

For LIBERO benchmarks, models are pruned with Wanda using a 15K calibration dataset drawn from the LIBERO fine-tuning corpus. For NaVILA, we use a 1K calibration dataset. The Memory-Matched Sparse Language Backbone prunes 75% of layers for OpenVLA and WorldVLA, and 81.3% of layers for NaVILA. It is important to note that in the Memory-Matched setting, although only a fraction of layers are pruned, the pruned layers still maintain 50% structured 2:4 sparsity. Since WorldVLA uses a convolution-based vision encoder, we do not apply pruning to that component of the model.

D.1 GLUESTICK Manipulation Task Performance Recovery

Model	Method	Success (%)	VRAM (GB)	ΔSuccess	ΔVRAM
OpenVLA	Full Dense	85.2	16.12	+0.0	+0.00
	Full Sparse	0.0	10.17	-85.2	-5.95
	Sparse Language Backbone	31.2	10.56	-54.0	-5.56
	75% Sparse Language Backbone	25.4	11.96	-59.8	-4.16
	GLUESTICK-200	49.0	11.18	-36.2	-4.94
	GLUESTICK-500	**60.2**	**11.97**	**-25.0**	**-4.15**
WorldVLA	Full Dense	88.4	16.24	+0.0	+0.00
	Sparse Language Backbone	3.4	10.16	-85.0	-6.08
	75% Sparse Language Backbone	5.0	12.16	-83.4	-4.08
	GLUESTICK-500	**47.8**	**12.14**	**-40.6**	**-4.10**

Table 11: **LIBERO Spatial.** Δ columns are relative to each model's Full Dense baseline on this benchmark. VRAM measured during inference on same hardware. Corrections stored in Int8.

Model	Method	Success (%)	VRAM (GB)	ΔSuccess	ΔVRAM
OpenVLA	Full Dense	85.8	16.12	+0.0	+0.00
	Full Sparse	13.4	10.17	-72.4	-5.95
	Sparse Language Backbone	50.8	10.56	-35.0	-5.56
	75% Sparse Language Backbone	49.0	11.96	-36.8	-4.16
	GLUESTICK-200	66.7	11.18	-19.1	-4.94
	GLUESTICK-500	**71.2**	**11.97**	**-14.6**	**-4.15**
WorldVLA	Full Dense	80.4	16.24	+0.0	+0.00
	Sparse Language Backbone	0.8	10.16	-79.6	-6.08
	75% Sparse Language Backbone	1.4	12.16	-79.0	-4.08
	GLUESTICK-500	**25.2**	**12.14**	**-55.2**	**-4.10**

Table 12: **LIBERO Object.** Δ columns are relative to each model's Full Dense baseline on this benchmark. VRAM measured during inference on same hardware. Corrections stored in Int8.

Model	Method	Success (%)	VRAM (GB)	ΔSuccess	ΔVRAM
OpenVLA	Full Dense	77.0	16.12	+0.0	+0.00
	Full Sparse	0.8	10.17	-76.2	-5.95
	Sparse Language Backbone	20.0	10.56	-57.0	-5.56
	75% Sparse Language Backbone	20.8	11.96	-56.2	-4.16
	GLUESTICK-200	30.4	11.18	-46.6	-4.94
	GLUESTICK-500	**47.0**	**11.97**	**-30.0**	**-4.15**
WorldVLA	Full Dense	81.0	16.24	+0.0	+0.00
	Sparse Language Backbone	21.0	10.16	-60.0	-6.08
	75% Sparse Language Backbone	21.6	12.16	-59.4	-4.08
	GLUESTICK-500	**45.2**	**12.14**	**-35.8**	**-4.10**

Table 13: **LIBERO Goal.** Δ columns are relative to each model's Full Dense baseline on this benchmark. VRAM measured during inference on same hardware. Corrections stored in Int8.

Model	Method	Success (%)	VRAM (GB)	ΔSuccess	ΔVRAM
OpenVLA	Full Dense	55.8	16.12	+0.0	+0.00
	Full Sparse	0.0	10.17	-55.8	-5.95
	Sparse Language Backbone	12.4	10.56	-43.4	-5.56
	75% Sparse Language Backbone	11.4	11.96	-44.4	-4.16
	GLUESTICK-200	16.2	11.18	-39.6	-4.94
	GLUESTICK-500	**26.6**	**11.97**	**-29.2**	**-4.15**
WorldVLA	Full Dense	55.2	16.24	+0.0	+0.00
	Sparse Language Backbone	0.0	10.16	-55.2	-6.08
	75% Sparse Language Backbone	0.0	12.16	-55.2	-4.08
	GLUESTICK-500	**0.0**	**12.14**	**-55.2**	**-4.10**

Table 14: **LIBERO Long.** Δ columns are relative to each model's Full Dense baseline on this benchmark. VRAM measured during inference on same hardware. Corrections stored in Int8. It is worth noting that WorldVLA completely collapsed after pruning on LIBERO Long, producing invalid action tokens and failing to generate meaningful outputs. Although task success rates did not improve, GLUESTICK was able to restore the model to producing more valid outputs.

D.2 GLUESTICK Navigation Task Performance Recovery

Full Dense trajectories remain short and goal-directed, while Full Sparse trajectories are substantially longer and terminate farther from the goal, reflecting severe degradation in navigational ability (see Figure 8). In contrast, GLUESTICK-500 closely matches the Full Dense distribution, indicating that it restores not only success rates but also the efficiency and precision of navigation behavior.

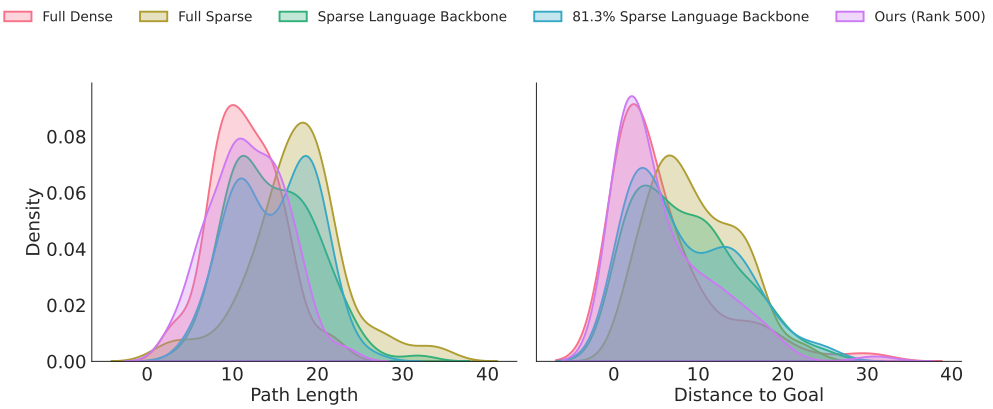


Figure 8: **Navigation Trajectory Quality.** Distribution of path lengths (left) and final distances to goal (right) for pruned NaVILA models and GLUESTICK on VLN-CE-Isaac.

Method	**Success** (↑)	**ΔUnsafe** (↓)	**PL** (↓)	**DG** (↓)	**ΔVRAM** (↓)
Full Dense	43.0	+0.0	11.7	5.9	+0.00
Full Sparse	0.0	+23.0	17.6	9.5	−5.74
Sparse Language Backbone	23.0	+2.0	14.8	8.5	−5.68
% Sparse Language Backbone	25.0	+12.0	14.9	8.4	−4.59
GLUESTICK-200	41.0	−1.0	12.5	6.5	−5.36
GLUESTICK-500	**44.0**	**−4.0**	**11.9**	**5.9**	**−4.60**

Table 15: **Navigation results.** Success denotes task success rate (%). ΔUnsafe and ΔVRAM are reported relative to Full Dense; lower values are better. All methods use the NaVILA model. PL = Path Length. DG = Final Distance to Goal.

D.3 Inference Speedup

We measure edge inference latency on an NVIDIA Jetson Thor and an NVIDIA Jetson Orin NX. Latency is reported as mean ± standard deviation of the end-to-end inference time. For OpenVLA, the end-to-end inference time corresponds to producing one full 7-DoF action; OpenVLA does not perform action chunking, so a single inference comprises both the prefill of the multimodal prompt and the autoregressive decode of all 7 action tokens. For WorldVLA, we use the default chunk size of 25 actions and report the wall-clock time to produce the entire chunk (prefill plus autoregressive decode of all 25 actions). All values are averaged over 100 timed iterations after 3 warm-up iterations.

We do not rely on `torch.sparse`; instead, we expose NVIDIA's `cuSPARSELt` 2:4 structured-sparse matmul kernels directly to Python through a small `pybind11` CUDA extension written against the C API. cuSPARSELt selects its sparse GEMM kernel at runtime by autotuning over ∼90 candidate algorithms via `cusparseLtMatmulSearch`, which is itself a non-trivial cost. Since every Linear layer in the model is evaluated at the same static shape on every iteration, we run this search once per unique shape, and cache the resulting plan.

We run the Jetson Thor with JetPack 7.1 and CUDA 13.2, and the Jetson Orin NX with JetPack 6.2 and CUDA 12.6. Both platforms use cuSPARSELt 0.8.1.1 and `torch.bfloat16`. All experiments are performed in MAXN performance mode with locked GPU clocks. The NVIDIA Jetson Orin NX has 16 GB of unified CPU/GPU memory; out of the box, the system reserves around 1.8 GB for background services, leaving a practical budget of roughly 13 GB. The stock Hugging Face loader combined with cuSPARSELt compression temporarily exceeds this budget during model loading before memory settles. To stay within the Orin NX's 13 GB budget, we replace the stock loader with a custom streaming loader that walks the `safetensors` shards, materializing each language-model `nn.Linear` directly on the GPU and applying cuSPARSELt compression one layer at a time to avoid exceeding peak memory.

E Analysis

E.1 VLA Component Sensitivity

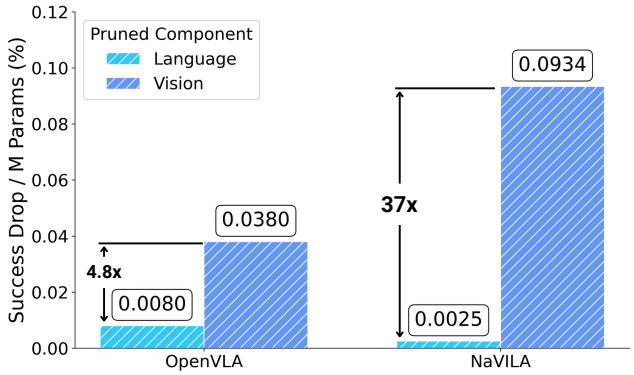


Figure 9: **Component Sensitivity**: Success-rate drop per million parameters when pruning only the language or vision backbone. Vision pruning is $4.8\times$ (OpenVLA) and $37\times$ (NaVILA) more damaging per parameter.

We investigate which VLA components are most sensitive to pruning. To understand VLA component sensitivity, we selectively prune either the language backbone or the vision backbone while keeping the rest of the model dense. For OpenVLA (7.5B total parameters: 89.4% in the language backbone, 9.7% in the vision backbone, and 0.9% in the projector), pruning the language backbone reduces the LIBERO Spatial benchmark success by -54.0%, while pruning the vision backbone reduces success by -27.8%. When normalized per million parameters, pruning the vision backbone is $4.75\times$ more damaging than pruning the language backbone (Figure 9).

We observe the same phenomenon in NaVILA (8.5B total parameters: 94.5% in the language backbone, 5.0% in the vision encoder, and 0.4% in the projector). Here, pruning the language backbone reduces success by -20.0%, while pruning the vision backbone reduces success by -40.0%. On a per-parameter basis, vision pruning is $37.5\times$ more damaging than language pruning (Figure 9). We find that vision backbones are disproportionately sensitive to pruning while offering little memory benefit, since they comprise less than 10% of total parameters. Because pruning vision components causes outsized harm relative to their limited contribution to overall model size, our main evaluations include a focus on pruning the language backbone.

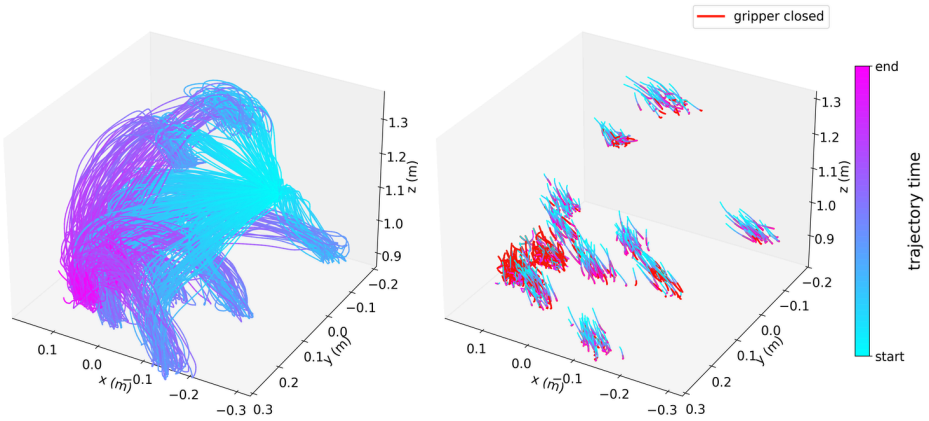


Figure 10: **Calibration Set Visualization.** LIBERO Spatial calibration trajectories. Full trajectories **(left)**. 5% window around the first gripper-closing event, with red segments marking closed-gripper states **(right)**.

E.2 Calibration Set Selection

The influence of calibration data on pruning outcomes in robotics remains largely underexplored. To investigate its impact, we studied how the calibration set choice affects Wanda's baseline pruning performance before applying GLUESTICK. Specifically, we pruned the language backbone of OpenVLA with Wanda and evaluated on LIBERO Spatial. We provided Wanda with 15K states from the LIBERO fine-tuning corpus. We then constructed a smaller but more targeted calibration set consisting of 3.5K states within a 5% window around the first gripper-closing event (Figure 10). Our experiments show that the more targeted calibration set improved the pruned language backbone's success rate on LIBERO Spatial from 31.2% to 33.4%, a gain of about 2%. Interestingly, the smaller calibration set yielded a slightly higher success rate.

E.3 Singular Value Selection

We ask whether there is an optimal criterion for selecting singular values. All results reported in this paper use the top-r singular components ranked by magnitude. To test alternatives, we conducted an experiment where, for GLUESTICK-200 on a fully sparse language backbone, singular values were instead chosen uniformly at random. In this setting, the model recovered only $\sim$10% of the task success achieved by magnitude-based selection. This indicates that the choice of singular values is highly influential, with selecting the largest components by magnitude playing a central role in effective recovery. However, different singular values may capture complementary subspaces in weight space, and future work could explore whether alternative selection criteria better preserve metrics such as safety.

E.4 Activation-Aware Variant of GLUESTICK

We explored an activation-aware variant of GLUESTICK, in which input activations steer which components of the gap are recovered. Using Wanda's calibration activation norms $c_j = \|X_j\|_2$ and $D = \mathrm{diag}(c)$, we scale the gap matrix as $\widetilde{W}_{\mathrm{gap}} = W_{\mathrm{gap}} D$, take its top-$r$ SVD, and undo the scaling via $V_r = D^{-1}\widetilde{V}_r$. This changes the recovered subspace: the cosine similarity between the two corrections is 0.86 at rank 500 and 0.78 at rank 200. However, the variant performs slightly worse on LIBERO Spatial, reaching 59.0% vs. 60.2% (rank 500) and 47.6% vs. 49.0% (rank 200). Its offline cost is nearly identical (2.375 vs. 2.366 s/layer, excluding Wanda's already-computed calibration activations), and its inference cost is unchanged (parameters, VRAM, and latency). Other activation-aware selection criteria remain a direction for future work.

E.5 Manipulation vs. Navigation Tolerance

In the navigation domain, small deviations in the robot's trajectory have minimal impact on final success. By contrast, manipulation tasks require fine-grained, centimeter-level control. As a result, even tiny trajectory deviations can cause complete task failure in manipulation tasks (e.g., slightly off from the grasp point of an object). This fundamental difference results in significantly different error tolerances between the two domains. To make this intuition concrete, we compared the average difference in trajectory length between successful and unsuccessful episodes in each domain.

Domain	Avg. Path-Length Differences
Manipulation	0.018 m
Navigation	3.292 m

Table 16: **Average path-length difference between successful and failed episodes** in manipulation and navigation tasks. Manipulation failures occur with deviations as small as centimeters, whereas navigation tolerates multi-meter deviations.

Table 16 demonstrates that manipulation success relies on precise control, where deviations as small as a centimeter can lead to failure. In contrast, navigation operates in a low-precision control regime, where deviations up to several meters can still lead to successful outcomes.

E.6 Model Parameter Analysis

We apply GLUESTICK to linear layers because linear layers generally make up the vast majority of parameters in recent VLA models as shown in Table 17.

Model	Total Parameters	Linear Parameters	Conv. Parameters
OpenVLA	7,541,237,184	7,407,513,280 (98.23%)	1,281,664 (0.02%)
WorldVLA	7,042,582,528	6,744,440,832 (95.77%)	27,310,848 (0.39%)
NaVILA	8,494,180,416	7,962,922,816 (93.75%)	678,528 (0.01%)

Table 17: **Parameter composition of recent VLA models.** Across OpenVLA, WorldVLA, and NaVILA, the vast majority of parameters (**93–98%**) are contained in linear layers, while convolutional components represent well under 1% of total model size.

E.7 Weight-Only Decomposition

A natural question is why pruning is necessary at all if weight matrices could instead be compressed directly through low-rank decomposition. In other words, why not compress weights directly with SVD and avoid pruning altogether? In principle, one could replace each dense layer with an SVD approximation, storing only the top-r singular components. To test this, we conducted experiments where OpenVLA weights were approximated at various SVD ranks (without pruning). On LIBERO Spatial, this setting achieved a 0% success rate across nearly all experiments, indicating that low-rank approximations alone are insufficient to preserve the functionality of VLA models. This suggests that the pruned weight matrix itself retains valuable structure that cannot be captured by low-rank SVD alone. GLUESTICK leverages this by preserving the pruned weights (and structured sparsity benefits) while using SVD only to reintroduce lost directions, nudging the model back toward a more performant region of weight space.

Rank	200	400	500	800	1000	2000	2500
Success Rate (%)	0	0	0	0	0	0.4	21.0

Table 18: Effect of replacing OpenVLA weights with an approximation using SVD (no pruning).

In Table 18, all entries are computed without applying any pruning. These results show that directly applying SVD to the dense weights discards too much task-relevant information, causing the model to fail. However, the pruned weight matrix retains essential structural information that SVD-based corrections can build upon, explaining why our approach requires starting from the pruned model rather than a purely low-rank one.

E.8 FLOP Analysis

As displayed in Table 19, sparse models reduce compute substantially, and GLUESTICK introduces only a modest FLOP increase relative to the sparse baseline.

Method	Total FLOPs (T)
Full Dense	29.32
50% 2:4 Pruned OpenVLA	16.54
50% 2:4 Pruned OpenVLA + GLUESTICK-200	18.51
50% 2:4 Pruned OpenVLA + GLUESTICK-500	21.47

Table 19: **FLOP analysis for a single inference pass through OpenVLA.** FLOP counters for the following operations: *aten.convolution*, *aten.add*, *aten.addmm*, *aten._scaled_dot_product_flash_attention*, *aten.mm*, and *aten.bmm*. In these experiments the language backbone of OpenVLA was pruned with Wanda.

F Discussion

F.1 GLUESTICK on Long-Horizon Tasks

In our main results, GLUESTICK recovers less on LIBERO Long than on LIBERO Spatial, which might suggest that it struggles with long-horizon tasks. However, the dense models themselves already perform relatively poorly on LIBERO Long ($\sim$55%), and pruning degrades them even more severely. For example, pruned WorldVLA no longer produces valid action tokens (Table 14). To test whether horizon is the limiting factor, we evaluate OpenVLA-OFT [95], a stronger VLA, where GLUESTICK improves success on LIBERO Long from 61.2% to 84.4%. This suggests that the gains of GLUESTICK depend more on the strength of the underlying VLA than on task horizon. Furthermore, VLN-CE-Isaac is itself a long-horizon benchmark, averaging 1828.2 steps per episode compared to 391.1 for LIBERO Long ($4.7\times$ longer), yet GLUESTICK fully recovers success there (Table 4). Together, these results suggest that GLUESTICK is not inherently limited by task horizon.

F.2 GLUESTICK and LoRA Differences

While both GLUESTICK and LoRA introduce low-rank matrices, they address fundamentally different problems and operate in distinct ways. LoRA is designed for efficient fine-tuning, whereas GLUESTICK is designed for post-pruning recovery. Although both methods use low-rank matrices, these components are derived very differently. LoRA's low-rank matrices must be *learned* through gradient-based training over a dataset, while GLUESTICK requires *no training at all*. Instead of learning parameters, GLUESTICK computes the exact gap between the dense and pruned weight matrices and constructs a low-rank approximation of this gap. Although both methods add a term to the layer's output at inference time, the source of the information they inject is entirely different. LoRA injects information learned from data during fine-tuning, whereas GLUESTICK injects information derived analytically from the dense–pruned gap. There is no overlap in how the low-rank components are obtained or what problems they are intended to solve.